Lista de Compras

Trabalho Bimestral — Paradigma de Programação

Aplicação desktop em Java com JavaFX, Scene Builder e Programação Orientada a Objetos

Integrantes
Aguinaldo Alves
Gustavo Pereira
Gustavo Miguel
João Pedro

Novembro de 2026

1. Integrantes e divisão do trabalho

Integrante	Parte do sistema	Histórias (BDD)
Aguinaldo Alves	Model (`Cadastro` e filhas, `ItemLista` , `Repositorio`), `Persistencia` (JSON), `Navegador` com passagem de dados, janelas de confirmação	NF001 (RN074–RN077)
Gustavo Pereira	Menu principal, Sair do sistema e tela Cadastrar lista de compras	H01–H05, H18
Gustavo Miguel	Telas Cadastrar estabelecimento e Cadastrar produto (com foto e imagem padrão)	H06–H13
João Pedro	Tela Criar lista de compras: incluir, consultar, editar e excluir itens, com totalizadores	H14–H17

2. Funcionamento do sistema

O **Lista de Compras** ajuda o usuário a organizar as compras da casa. Ele cadastra as **listas** (por exemplo, "Compras da semana"), os **estabelecimentos** onde costuma comprar, com os dias de promoção, e os **produtos**, com foto, descrição e unidade de medida. Depois ele **monta a lista**: escolhe a lista e, se quiser, o estabelecimento, e inclui os produtos com quantidade e valor. O sistema calcula o subtotal de cada item, o total de itens e o valor total da compra.

Em todas as telas de cadastro o usuário pode incluir, consultar, editar e excluir. Exclusões e a saída do sistema sempre pedem confirmação. Um cadastro que já está sendo usado em algum item não pode ser excluído. Todos os dados ficam gravados no arquivo `dados/lista_compras.json` e voltam quando o sistema é aberto de novo.

A aplicação roda offline, em Windows com Java instalado, para um único usuário e sem login.

Telas

Tela	Papel no trabalho de Java	O que faz
Menu principal	Navegação	Cinco opções fixas: cadastrar lista, cadastrar estabelecimento, cadastrar produto, criar lista e sair
Cadastrar lista de compras	**Cadastro + Consulta**	Inclui, edita e exclui listas. A tabela mostra cada lista com a quantidade de produtos e o valor total, e tem o botão **Montar**
Cadastrar estabelecimento	**Cadastro + Consulta**	Nome e dias de promoção (Dom a Sáb). A tabela mostra os dias na ordem da semana
Cadastrar produto	**Cadastro + Consulta**	Foto (arquivo JPEG/PNG ou link), descrição e unidade. Os produtos aparecem em cartões; sem foto, aparece a imagem padrão
Criar lista de compras	**Movimentação + Consulta**	Inclui, edita e exclui itens da lista, com subtotal, total de itens e valor total

3. Estrutura das classes

O projeto segue o padrão **MVC**:

Camada	Pacote / pasta	Conteúdo
Model	`br.com.listacompras.model`	`Cadastro` (abstrata), `ListaCompra`, `Estabelecimento`, `Produto`, `ItemLista`, `UnidadeMedida` (enum), `Repositorio`, `Persistencia`, `ValidacaoException`
View	`src/main/resources/view`	Cinco arquivos FXML feitos no Scene Builder, mais o `css/estilo.css`
Controller	`br.com.listacompras.controller`	Um controller por tela, mais `ExclusaoCadastro` (exclusão comum às três telas de cadastro)
Utilitários	`br.com.listacompras.util`	`Navegador` (troca de tela), `Mensagem` (avisos e confirmações), `BotoesLinha` (botões na linha), `Formatador` (R$ e quantidades), `Encerramento` (regras de saída)

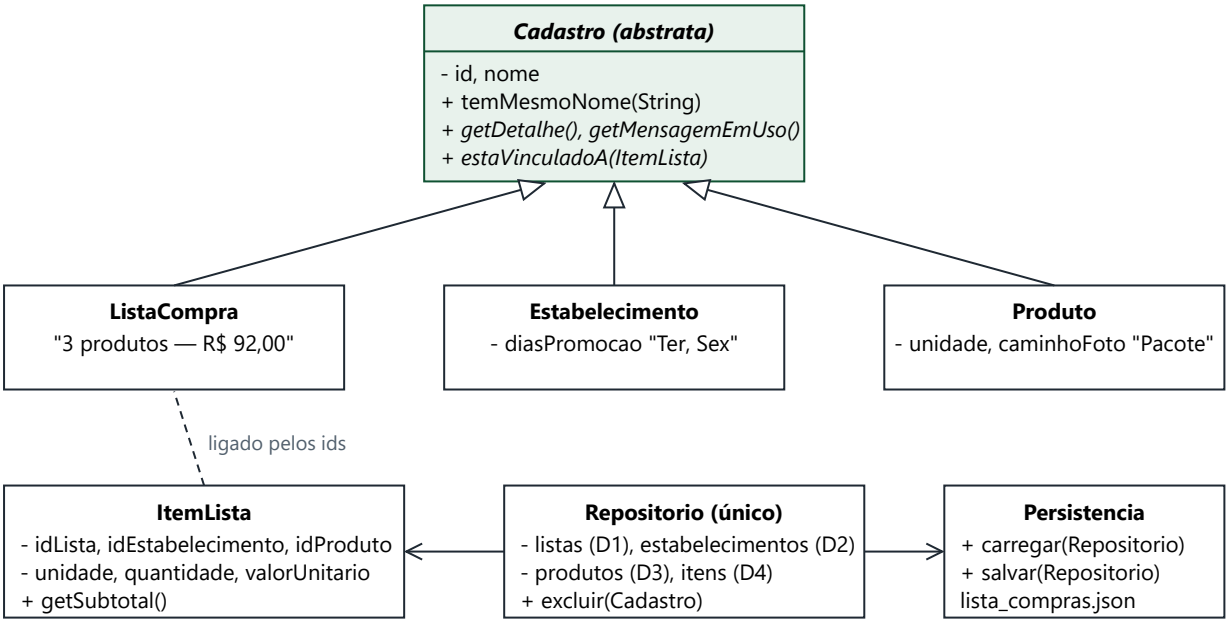


Figura 1 — Classes do Model. O triângulo indica herança; as setas indicam uso.

Depósitos do DFD e classes

Os quatro depósitos de dados do DFD (trabalho de Processo de Software) viraram as quatro listas do `Repositorio` e as quatro partes do arquivo JSON:

Depósito do DFD	Classe	Lista no Repositorio	Chave no JSON
D1 Listas de compras	`ListaCompra`	`listas`	`"listas"`
D2 Estabelecimentos	`Estabelecimento`	`estabelecimentos`	`"estabelecimentos"`
D3 Produtos	`Produto`	`produtos`	`"produtos"`
D4 Itens da lista	`ItemLista`	`itens`	`"itens"`

4. Conceitos de Programação Orientada a Objetos

Conceito	Onde aparece no projeto
Classe e objeto	Cada lista, estabelecimento, produto e item cadastrado é um objeto das classes do Model
Encapsulamento	Todos os atributos são `private` e só são acessados por getters e setters. O `Repositorio` devolve listas que não podem ser alteradas por fora (`Collections.unmodifiableList`), então toda mudança passa pelas regras de negócio
Abstração	`Cadastro` é `abstract`: não existe um "cadastro genérico", só listas, estabelecimentos e produtos
Herança	`ListaCompra`, `Estabelecimento` e `Produto` estendem `Cadastro`. `ValidacaoException` estende `RuntimeException`. `App` estende `Application` do JavaFX
Polimorfismo	`getDetalhe()`, `getMensagemEmUso()` e `estaVinculadoA()`, chamados sobre uma variável do tipo `Cadastro` (seção 7)
Enumeração	`UnidadeMedida`: a unidade só pode ser uma das cinco previstas (RN038)
Tratamento de exceções	As regras de negócio lançam `ValidacaoException` com a mensagem do BDD; os controllers capturam e mostram na tela
Padrão Singleton	`Repositorio.getInstancia()`: um único repositório para o sistema inteiro

5. Passagem de dados entre telas

As telas trabalham com **os mesmos objetos**. O caminho de um dado é este:

Etapa	Como acontece
Criado	Na tela Cadastrar lista, o controller chama `repositorio.cadastrarLista("Compras da semana")`, que valida e cria o objeto `ListaCompra`
Armazenado	O objeto fica na lista `listas` (D1) do único `Repositorio` e é gravado no JSON na hora (RN074)

Enviado	O botão **Montar** da linha abre a tela Criar lista e entrega o objeto para ela:
	`CriarListaController criarLista = Navegador.abrir(btnVoltar, "/view/CriarLista.fxml");`
	`criarLista.setListaSelecionada(lista);`
	O `Navegador.abrir` usa um objeto `FXMLLoader`, carrega a tela nova e devolve o controller dela com `getController()`

Recuperado	`setListaSelecionada` coloca o objeto na lista suspensa, e a tela já mostra os itens daquela lista. As listas suspensas de lista, estabelecimento e produto mostram os objetos guardados no Repositorio
Alterado	Ao incluir, editar ou excluir itens, o Repositorio altera os objetos `ItemLista` (D4), ligados à lista pelo id
Refletido	Ao voltar ao Cadastrar lista, a coluna DETALHES chama `lista.getDetalhe()`, que calcula na hora "3 produtos — R$ 92,00" a partir dos itens. A lista também não pode mais ser excluída: "Não é possível excluir: a lista possui itens vinculados"

A opção "Criar lista de compras" continua no menu, como exige a RN078. Nesse caso o usuário escolhe a lista na própria tela.

6. Herança

A classe pai `Cadastro` guarda o que as três telas de cadastro têm em comum: o `id`, o `nome` e a regra de nome repetido. As filhas chamam o construtor do pai com `super(id, nome)` e acrescentam só o que é delas.

```
public abstract class Cadastro {
    private int id;
    private String nome;

    public boolean temMesmoNome(String outroNome) {
        return nome.trim().equalsIgnoreCase(outroNome.trim());
    }
    public abstract String getDetalhe();
    public abstract String getMensagemEmUso();
    public abstract boolean estaVinculadoA(ItemLista item);
}

public class Estabelecimento extends Cadastro {
    private List<DayOfWeek> diasPromocao;
    public Estabelecimento(int id, String nome, List<DayOfWeek> dias) {
        super(id, nome);                // o pai guarda id e nome
        this.diasPromocao = organizarDias(dias);
    }
}
```

Benefício: a regra "o nome não pode se repetir, ignorando maiúsculas, minúsculas e espaços" (RN007, RN022 e RN037) foi escrita **uma única vez**, em `temMesmoNome()`, e as três classes a herdam. O método `recusarNomeRepetido()` do Repositorio recebe `List<? extends Cadastro>` e serve para os três tipos.

7. Polimorfismo

Onde	Classes	O que varia
Coluna DETALHES / cartão do produto: `cadastro.getDetalhe()`	ListaCompra, Estabelecimento, Produto	Lista: "3 produtos — R$ 92,00"; Estabelecimento: "Ter, Sex" ou "Sem dias de promoção"; Produto: "Pacote"
Exclusão: `ExclusaoCadastro.excluir(Cadastro)` → `repositorio.excluir(Cadastro)`	As três	Se estiver em uso, cada classe devolve a própria frase do BDD em `getMensagemEmUso()` (H05, H09 e H13)
Verificar se está em uso: `cadastro.estaVinculadoA(item)`	As três	A lista compara `idLista`, o estabelecimento compara `idEstabelecimento` e o produto compara `idProduto`

```
// Um único método serve para lista, estabelecimento e produto
public void excluir(Cadastro cadastro) {
    if (estaEmUso(cadastro)) {
        throw new ValidacaoException(cadastro.getMensagemEmUso());
    }
    ...
}
```

Benefício: as três telas usam o mesmo código de exclusão. Quem chama não precisa de `if` para saber o tipo do objeto: o Java chama a versão do método da classe real em tempo de execução. Um quarto tipo de cadastro só precisaria implementar os três métodos.

8. Fluxo da aplicação

1. O `Main` chama o `App`, que carrega o JSON (ou cria o arquivo vazio) e abre o Menu.
2. **Cadastro:** o usuário cadastra o estabelecimento "Mercado Central" (Ter, Sex), os produtos e a lista "Compras da semana".
3. **Consulta:** as tabelas e os contadores mostram os registros; a lista aparece com "0 produtos — R$ 0,00".
4. **Movimentação:** o botão Montar abre o Criar lista com a lista escolhida; o usuário inclui os itens e vê os totais (12 itens, R$ 92,00).
5. **Consulta de novo:** de volta ao Cadastrar lista, a mesma linha mostra "3 produtos — R$ 92,00".
6. **Sair:** confirma, grava o arquivo e fecha. Ao abrir de novo, os dados continuam lá.

> **Por que a Consulta não é uma tela separada:** o documento de requisitos do grupo (trabalho de Processo de Software) define que o menu tem exatamente cinco opções (RN002) e que a consulta é exibida "na própria tela de cadastro, sem exigir uma ação extra do usuário" (RN010). Por isso a Consulta exigida pelo trabalho de Java fica embutida nas telas de cadastro (tabelas e cartões) e na tabela de itens do Criar lista (RF015).

9. Gravação dos dados em JSON

A classe `Persistencia` usa a biblioteca **Gson** (dependência no `pom.xml`) para transformar os objetos em JSON e o JSON de volta em objetos.

Regra	Como foi feito
RN074 — gravar a cada operação	Todo método do Repositorio que altera dados termina chamando `gravar()`
RN075 — carregar ao iniciar	`App.start()` chama `Repositorio.ativarGravacao(...)` antes de abrir o menu
RN076 — criar o arquivo vazio	Se `dados/lista_compras.json` não existe, ele é criado com as quatro listas vazias, sem mensagem de erro
RN077 — arquivo corrompido	O sistema avisa o usuário e abre sem dados, e guarda uma cópia `lista_compras.json.corrompido` para não perder o conteúdo
RN071 — gravar ao sair	`Encerramento.confirmarSaida()` grava o arquivo antes de fechar

```
{
  "listas": [ { "id": 1, "nome": "Compras da semana" } ],
  "estabelecimentos": [ { "diasPromocao": [ "TUESDAY", "FRIDAY" ], "id": 1, "nome": "Mercado
Central" } ],
  "produtos": [ { "unidade": "PACOTE", "id": 1, "nome": "Arroz branco 5kg" } ],
  "itens": [ { "id": 1, "idLista": 1, "idEstabelecimento": 1, "idProduto": 1,
               "unidade": "PACOTE", "quantidade": 2.0, "valorUnitario": 24.9 } ]
}
```

10. Testes

Foram feitos dois testes automáticos, que ficam em `src/test/java` e rodam pelo IntelliJ (botão direito no arquivo > Run):

Teste	O que verifica	Resultado
`TesteModel`	Regras do Model sem abrir telas: validações, duplicidade, ordem dos dias, cálculos R$ 92,00 → R$ 103,00 → R$ 82,80	52 de 52 OK
`TesteTelasBDD`	Abre as telas de verdade e segue os cenários H01 a H18 com os dados e as mensagens do BDD, clicando nos botões (inclusive Editar, Excluir e Montar dentro das linhas)	91 de 91 OK

História	Cenários conferidos	Situação
H01 Menu	c1, c2, c4	OK (c3 não se aplica*)
H02 a H05 Listas	todos	OK
H06 a H09 Estabelecimentos	todos	OK
H10 a H13 Produtos	todos, incluindo o aviso ao trocar a unidade (H12 c2)	OK
H14 a H17 Itens	todos, incluindo R$ 49,80, 12 itens / R$ 92,00, R$ 103,00, R$ 9,20 e R$ 82,80	OK
H18 Encerrar	c1, c2, c3, mais o aviso de cadastro em andamento (RN072)	OK

* H01 c3 / RN004 ("opção inválida"): em interface gráfica o menu só tem botões, então não existe como informar uma opção que não está no menu.

11. Como executar

1. Abrir a pasta `Lista_Compras` no IntelliJ IDEA (projeto Maven; o IntelliJ baixa JavaFX 21 e Gson sozinho).
2. Executar a classe `org.example.Main`, ou rodar `mvn javafx:run`.
3. Na primeira execução é criada a pasta `dados` com o arquivo `lista_compras.json`.

A classe `Main` não herda de `Application` de propósito: assim o JavaFX funciona mesmo estando no classpath, sem o erro "JavaFX runtime components are missing".

Apêndice — Roteiro da apresentação

#	Item	O que mostrar	Quem
1	Cadastro	Cadastrar "Mercado Central" (Ter, Sex); "Arroz branco 5kg" (Pacote), "Leite integral" (Litro), "Sabonete" (Unidade); a lista "Compras da semana"	G. Miguel / G. Pereira
2	Consulta	Tabelas e contadores; a lista com "0 produtos — R$ 0,00"; dias na ordem da semana; imagem padrão	G. Pereira / G. Miguel
3	Movimentação	Clicar em **Montar**, escolher o Mercado Central e incluir arroz 2 × 24,90, leite 6 × 5,50 e sabonete 4 × 2,30	João Pedro
4	Validação	Quantidade 0 → "A quantidade deve ser maior que zero"; nome repetido → "Já existe uma lista com esse nome"	João Pedro
5	Atualização	Totais 12 itens e R$ 92,00; editar o leite para 8 → R$ 103,00	João Pedro
6	Consulta de novo	Voltar ao Cadastrar lista: "3 produtos — R$ 103,00"; tentar excluir e ver o bloqueio	G. Pereira
7	Passagem de dados	No código: `Navegador.abrir` devolvendo o controller e `setListaSelecionada(lista)`; o Repositorio único	Aguinaldo
8	Objetos e JSON	Sair, abrir de novo e ver os dados; abrir o `lista_compras.json`	Aguinaldo
9	Herança e polimorfismo	`Cadastro`, `getDetalhe()`, `getMensagemEmUso()` e `ExclusaoCadastro`	Aguinaldo
10	Scene Builder	Abrir um FXML (por exemplo `CriarLista.fxml`) e mostrar os `fx:id` ligados ao controller	Todos